

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE FOUR

Vector Stores

Beginner to Industry Level • Assumes only basic Python

MODULE FOUR

Vector Stores

A knowledge base is only useful if it can be searched quickly. This module is about the database that makes RAG practical at scale: the vector store. You will learn what a vector database is and what it stores, why comparing a query against every vector one by one collapses at scale, and how the two great indexing strategies — IVF and HNSW — make similarity search fast by trading a sliver of accuracy for enormous speed. You will master the create, read, update, and delete operations every vector store supports, tour the six databases you will meet in industry — FAISS, ChromaDB, Pinecone, Weaviate, Milvus, and Qdrant — and learn how a production vector database is architected.

WHERE WE ARE

Module 3 turned every chunk into an embedding — a point on the "map of meaning." This module is the **store** step of the indexing pipeline: where those vectors live, how they are indexed for fast search, and how you create, read, update, and delete them. With Module 4 complete, the entire indexing phase is done. Module 5 then begins the query phase by searching the store you build here.

4.1 Introduction

At the end of Module 3 you had a pile of vectors — one per chunk — and a way to measure the similarity between any two of them. That is enough to *understand* retrieval but not enough to *build* it. A pile of vectors sitting in a Python list has two problems.

The first problem is **persistence**. A Python list lives in memory and vanishes the moment your program exits. Embedding a large corpus can take many minutes and many API calls; you cannot afford to throw that work away every time the process restarts. The vectors need a home that survives.

The second problem, and the deeper one, is **speed at scale**. Module 3's similarity search said "compare the query vector against *every* stored vector." With 100 chunks that is trivial. With 10 million chunks — an entirely normal size for an enterprise knowledge base — comparing against every vector on every single query is far too slow. A user will not wait several seconds for a chatbot to think.

The solution to both problems is a **vector store** (also called a **vector database**): a specialised database built from the ground up to *store* embeddings durably and to *search* them by similarity extremely fast — even across millions or billions of vectors. The trick that makes the search fast is called **indexing**, and understanding two indexing strategies — **IVF** and **HNSW** — is the intellectual core of this module.

This module gives the vector store the serious treatment it deserves. We will explain what a vector database stores and how it differs from a traditional database, dissect the brute-force baseline and why it fails, build a clear intuition for IVF and HNSW indexing, cover the four CRUD operations every store supports, survey the six vector databases you will encounter in real projects, and finish by writing real code against FAISS and ChromaDB.

4.2 Why This Topic Matters

WHY THIS MATTERS

The vector store is the component that decides whether your RAG system is a toy or a product. Get it right and retrieval is fast, durable, and scalable. Get it wrong and you face one of two failures: a system that is far too slow for real users, or a system that silently returns the wrong chunks because its index was tuned for speed at the expense of accuracy. Every "which vector database should we use?" decision — one of the most common questions in real RAG projects — rests on the concepts in this module.

Four concrete reasons this module repays close study:

1. **It is where the indexing phase becomes real and durable.** Loading, splitting, and embedding all produce intermediate data. The vector store is the *artifact* — the finished, persistent, searchable knowledge base.
2. **Indexing strategy is a genuine engineering trade-off.** IVF versus HNSW, and their parameters, trade speed, memory, and accuracy against each other. Understanding the trade-off is the difference between configuring a database and guessing at it.
3. **CRUD operations are how a knowledge base stays alive.** Real knowledge bases change. Documents are added, corrected, and removed. Knowing how to update and delete vectors — without rebuilding the whole index — is essential operational knowledge.
4. **Database selection is a recurring interview and project question.** "Why FAISS for a prototype but a managed database in production?" is exactly the kind of question that separates someone who has read about RAG from someone who has shipped it.

4.3 A Beginner-Friendly Explanation: What Is a Vector Store?

Let us build the idea plainly.

You already know what a normal database does. A traditional database — think of a spreadsheet or an SQL table — stores rows of structured data and lets you search by **exact match**: "find the customer whose `id` equals 4471," or "find all orders where `status` equals 'shipped'." It is brilliant at exact lookups and useless at anything fuzzy. It cannot answer "find the customer most *similar* to this one."

A **vector store** is a different kind of database, built for a different kind of question. It stores **vectors** — the embeddings from Module 3 — and its core skill is **similarity search**: "given this query vector, find the k stored vectors that are *closest* to it." It does not look for exact matches; it looks for *nearness*.

What does a vector store actually hold? For each chunk in your knowledge base, it stores three things bundled together:

- **The vector** — the embedding, the list of numbers representing the chunk's meaning. This is what gets searched.
- **The original text** — the actual chunk text. This is what gets returned and ultimately fed to the LLM. (A vector alone is not human-readable; you must keep the text.)
- **The metadata** — the dictionary from Module 2 (source, page, doc type, and so on). This is what enables citations and filtering.

So a single record in a vector store is a triple: `(vector, text, metadata)`. When you "search" the store, you compare your query vector against all the stored vectors, find the nearest ones, and the store hands you back their text and metadata.

DEFINITION

A **vector store** (or **vector database**) is a database specialised for storing high-dimensional vectors and performing fast **similarity search** — finding the stored vectors nearest to a query vector. Alongside each vector it stores the original text and its metadata. It is the persistent, searchable home of a RAG knowledge base.

The one capability that makes a vector store more than "a place to keep vectors" is **indexing** — an internal organisation of the vectors that lets the store find nearest neighbours *without* checking every vector. That is the subject of Section 4.5, and it is what this module is really about.

4.4 Real-World Analogy: The Organised Warehouse

REAL-WORLD ANALOGY

Imagine an enormous warehouse holding a million products, and your job is to find the products most *similar* to one a customer is holding.

The naive method — **brute force** — is to walk past every single shelf, examining all one million products, comparing each to the customer's item. It is perfectly accurate: you will definitely find the best matches. But for a million products it takes all day. This is searching *without an index*.

A smart warehouse manager instead **organises the warehouse into zones** — electronics here, kitchenware there, clothing over there — with a map at the entrance. Now, to find products similar to a customer's kettle, you read the map, walk straight to the kitchenware zone, and search only there. You examined a few thousand products instead of a million. You were slightly less thorough — a stray kettle misfiled in electronics would be missed — but you were a thousand times faster, and in practice you found excellent matches. This zoning is an **index**.

IVF indexing is exactly this "divide the warehouse into zones" idea. **HNSW** indexing is a different smart strategy: a network of signposts that let you *navigate* straight to the right region by following "this way to similar items" arrows.

This analogy carries the entire module:

In the analogy	In a vector store
A product on a shelf	A stored vector (with its text and metadata)
The customer's item	The query vector
Walking past every shelf	Brute-force (flat) search — exact but slow
Organising into zones with a map	An index (IVF or HNSW)
Searching only the relevant zones	Approximate Nearest Neighbour search
A misfiled item occasionally missed	The small accuracy cost of approximate search
Thousand-fold speed-up	Why indexing is non-negotiable at scale

Hold this picture. Indexing trades a *tiny, controllable* amount of accuracy for a *massive* gain in speed — and that bargain is what makes RAG viable at real-world scale.

4.5 Core Concepts: The Search Problem

4.5.1 Brute-Force (Flat) Search

The simplest possible vector search is **brute-force search**, often called **flat** search. For a query vector, you compute its distance to *every single stored vector*, sort all the results, and return the top k .

Brute-force search has one great virtue: it is **exact**. It is guaranteed to find the true nearest neighbours, because it checked everything. For small collections — up to perhaps tens of thousands of vectors — it is also perfectly fast and is the right choice.

Its fatal flaw is that its cost grows **linearly** with the number of vectors. Doubling the corpus doubles the search time. At a million vectors each query is sluggish; at a hundred million it is hopeless. Brute force does not scale.

4.5.2 The Idea of Approximate Nearest Neighbour Search

The escape from brute force is a profound and slightly counter-intuitive idea: **give up on a perfect answer in exchange for a fast one**.

Approximate Nearest Neighbour (ANN) search does not promise to find the *exact* top- k nearest vectors. It promises to find vectors that are *almost certainly* among the nearest — extremely close to the true answer — while examining only a small fraction of the database. In the warehouse analogy, ANN is searching only the relevant zones: you might miss one occasionally misfiled item, but you are a thousand times faster and your results are still excellent.

The quality of an ANN search is measured by **recall**: the fraction of the true nearest neighbours that the approximate search actually found. A recall of 0.98 means the index found 98% of the genuinely closest vectors — usually more than good enough, because RAG retrieves several chunks and the LLM only needs the relevant ones, not a mathematically perfect ranking.

DEFINITION

Approximate Nearest Neighbour (ANN) search is a search strategy that finds vectors very likely to be among the true nearest neighbours of a query, while examining only a small portion of the database. It trades a small, controllable loss of accuracy (measured by **recall**) for a very large gain in speed. **Indexing** is the internal data structure that makes ANN search possible.

WHY THIS MATTERS

Beginners are often uneasy that ANN is "not exact." Reframe it: in RAG, the goal is not to win a mathematics competition for the single closest vector — it is to put genuinely relevant chunks in front of the LLM. An index that finds 49 of the true top-50 chunks, a thousand times faster than brute force, is not a compromise — it is the *correct engineering choice*. Recall is a dial you can turn: spend a little more search effort for higher recall, or less for more speed. Brute force is simply ANN with the dial pinned at "exact and slow."

4.5.3 The Three-Way Trade-off

Every indexing strategy juggles three quantities, and you cannot maximise all three at once:

- **Speed** — how fast a query returns.
- **Recall (accuracy)** — what fraction of the true nearest neighbours it finds.
- **Memory** — how much RAM the index consumes (and how long it takes to build).

The art of configuring a vector store is choosing where to sit in this trade-off for your particular workload. The two dominant indexing strategies — IVF and HNSW — make this trade-off in different ways, which is exactly why both exist.

4.6 Technical Deep Dive: Indexing Strategies**4.6.1 IVF — Inverted File Index**

IVF (Inverted File index) is the "organise the warehouse into zones" strategy, made precise.

How the index is built (once, at indexing time):

1. Run a clustering algorithm (k-means) over all the vectors. This partitions the entire vector space into a chosen number of clusters — say 1,000 of them. Each cluster has a **centroid**, a vector marking its centre.
2. Every stored vector is assigned to its nearest centroid. The index now maintains, for each centroid, a list of all the vectors belonging to that cluster. (This list-per-cluster structure is the "inverted file" the strategy is named after.)

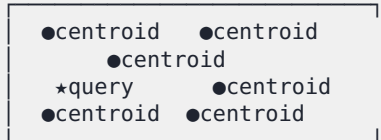
How a search works (at query time):

1. Compare the query vector against the *centroids only* — just 1,000 comparisons, not a million.
2. Pick the few centroids closest to the query. The number you pick is a tunable parameter, traditionally called **nprobe**.

3. Search — brute force — only the vectors inside those few selected clusters, and return the best.

IVF SEARCH (nprobe = 2)

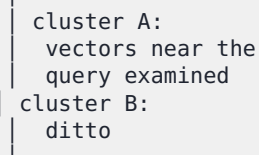
Step 1: query vs centroids only



pick the 2 nearest centroids
(clusters A and B)

→ all other clusters skipped entirely

Step 2: brute-force the
chosen clusters only



The `nprobe` parameter is the speed-versus-recall dial. A low `nprobe` (search 1–2 clusters) is very fast but may miss a true neighbour sitting just over a cluster border. A high `nprobe` (search many clusters) approaches brute-force accuracy but gives back the speed. IVF is memory-efficient and fast to build, and it shines on very large, fairly static datasets.

4.6.2 HNSW — Hierarchical Navigable Small World

HNSW (Hierarchical Navigable Small World) is a different and, today, the most widely used strategy. It is graph-based rather than cluster-based.

The intuition is a **multi-layer network of signposts**. Imagine navigating a country. The top layer is a sparse map of just a few major cities connected by long highways. Lower layers add more towns and shorter roads. The bottom layer contains every single village connected by local lanes. To travel somewhere, you start on the sparse top layer, take big highway jumps to get into the right region quickly, then drop down layer by layer, each time taking smaller, more precise steps, until you arrive.

How the index is built: every vector becomes a node in a graph and is connected to its nearby vectors by edges. The graph is built in **layers**: the bottom layer connects all vectors densely; each higher layer contains a random, progressively smaller subset with longer-range connections. The result is a structure that supports both big leaps and fine steps.

How a search works:

1. Start at an entry point on the **topmost, sparsest layer**.
2. Greedily walk to whichever neighbour is closest to the query — taking long-range "highway" jumps that cover ground fast.
3. When no neighbour on this layer is closer, **drop down one layer** and continue, now with shorter, more precise "local road" steps.
4. Repeat until the bottom layer, where the walk settles on the true nearest neighbours.

HNSW SEARCH – navigate down the layers



HNSW typically delivers **excellent recall at very high speed**, which is why it is the default index in most modern vector databases. Its cost is **memory**: storing the multi-layer graph of connections uses considerably more RAM than IVF, and building the graph is slower. Its key tunable parameters are **M** (how many connections each node keeps) and **ef** (how thoroughly the search explores) — both, again, dials on the speed-versus-recall trade-off.

4.6.3 Comparing the Indexing Strategies

Strategy	Core idea	Search speed	Recall	Memory use	Best suited to
Flat (brute force)	Compare against everything	Slow at scale	Perfect (exact)	Low	Small collections ($\leq$ tens of thousands)
IVF	Cluster space into zones; search nearest zones	Fast	High, tunable via nprobe	Low-moderate	Very large, fairly static datasets
HNSW	Multi-layer navigable graph	Very fast	Very high, tunable via ef	High	Most production workloads; the common default

INTERVIEW INSIGHT

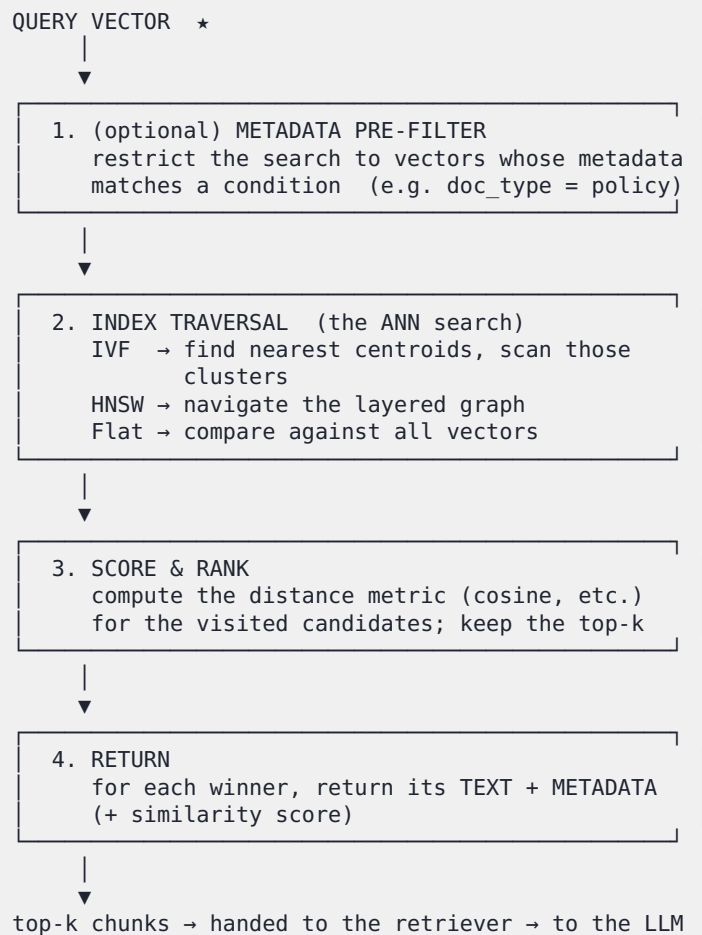
"Explain the difference between IVF and HNSW" is a standard senior-level question. The expected answer: IVF is *cluster-based* — it partitions the vector space into zones and at query time searches only the zones nearest the query (controlled by **nprobe**); it is memory-efficient and great for huge static datasets. HNSW is *graph-based* — it builds a multi-layer proximity graph and navigates from sparse long-range layers down to a dense local layer; it offers excellent recall and speed but uses more memory. The unifying point to state explicitly: **both are ANN methods that trade a little recall for a lot of speed, and both expose a parameter that lets you tune exactly where on that trade-off you sit.**

COMMON MISTAKE

Treating the index type as a setting you never think about, then being baffled when retrieval quality is poor. An index tuned aggressively for speed (very low `nprobe`, or a small HNSW `ef`) can *silently* return lower-recall results — the right chunk exists in the store but the search never visited it. No error is raised; answers simply get subtly worse. When debugging weak retrieval, the index configuration is a real suspect: temporarily compare against a flat (exact) search to see whether the index is the culprit.

4.7 Internal Architecture: Similarity Search Internals

Putting it together, here is what actually happens inside a vector store when the retriever asks it for the chunks nearest a query:



Three observations about this flow. First, **the index does not change what "nearest" means** — the distance metric from Module 3 (usually cosine) still defines similarity; the index only changes *which candidates get scored*, by cleverly skipping the ones that are obviously far away. Second, **metadata filtering happens around the vector search** — it shrinks the candidate set, which both sharpens precision and speeds the query (this is the mechanism behind Module 2's metadata filtering and Module 6's self-query retriever).

Third, **the store returns text and metadata, not just vectors** — because the LLM needs readable text and the system needs metadata for citations.

4.8 Vector Store CRUD Operations

A knowledge base is not built once and frozen. Documents are added, corrected, and retired. So a vector store must support the four classic database operations — **Create, Read, Update, Delete (CRUD)**. LangChain exposes these through a consistent interface across stores.

4.8.1 Create (Add)

Create means inserting new vectors. There are two patterns. You either build a store from a batch of documents all at once, or you add documents to an already-existing store.

```
# Pattern A – build a new store from a batch of chunks
vector_store = FAISS.from_documents(chunks, embeddings)

# Pattern B – add more chunks to an existing store later
new_ids = vector_store.add_documents(more_chunks)
```

Under the hood, "create" embeds each new chunk (if not already embedded) and inserts it into the index. Most stores return an **ID** for each inserted record — keep these IDs; they are how you will later update or delete that specific record.

4.8.2 Read (Similarity Search)

Read is the similarity search itself — the operation the whole module has been building toward.

```
# Return the k chunks most similar to the query
results = vector_store.similarity_search("How much annual leave?", k=4)

# Same, but also return each chunk's similarity score
scored = vector_store.similarity_search_with_score("How much annual leave?", k=4)
for doc, score in scored:
    print(score, doc.metadata, doc.page_content[:80])
```

`similarity_search` returns the top-*k* chunk `Document` objects. `similarity_search_with_score` additionally returns each chunk's distance/similarity score, which is essential for the **score thresholds** you will use in Module 5. A search can also take a metadata `filter`, as covered in Module 2.

4.8.3 Update

Update handles a chunk whose underlying content has changed. Importantly, you do not edit a vector in place — because changing the *text* changes its *meaning*, which means it must be **re-embedded**. An update is therefore conceptually a delete-then-insert keyed on the record's ID.

```
# Chroma exposes an explicit update by ID
vector_store.update_document(document_id="chunk_042", document=revised_chunk)

# Equivalent universal pattern: delete the old record, then add the new one
vector_store.delete(ids=["chunk_042"])
vector_store.add_documents([revised_chunk], ids=["chunk_042"])
```

This is exactly why Module 2 insisted on stable, consistent metadata and IDs: without a reliable handle on each record, you cannot target the right vector to update.

4.8.4 Delete

Delete removes vectors — for a retired document, a correction, or a compliance request ("erase everything from this source").

```
# Delete specific records by their IDs
vector_store.delete(ids=["chunk_042", "chunk_043"])
```

Deletion by ID removes individual records. To remove an entire document's worth of chunks, you find all chunk IDs that share that document's **source** metadata and delete them as a set — the incremental-update pattern from Module 2's pipeline.

BEST PRACTICE

Design your knowledge base around **stable, deterministic IDs** from the start — for example, an ID derived from the source file plus the chunk's position. With deterministic IDs, re-ingesting a changed document naturally overwrites its old chunks instead of creating duplicates, and updates and deletes become precise and safe. Random IDs assigned at insert time make the knowledge base far harder to keep correct over time.

4.9 The Vector Database Landscape

You will meet six vector databases repeatedly. They differ mainly along one axis: **how much infrastructure you manage yourself**, ranging from an in-process library to a fully managed cloud service.

- **FAISS** — *Facebook AI Similarity Search*. Not a database but a **library**: a highly optimised similarity-search engine that runs inside your Python process. No server, no network. It is in-memory, though you can save and load an index to disk. Brilliant for learning, prototyping, and embedding search into a single application. It has no built-in metadata filtering layer or multi-user serving. **This book's default for examples**, because it is free, fast, and zero-setup.
- **ChromaDB (Chroma)** — a lightweight, open-source vector database designed for developer friendliness. It runs **embedded** (in-process, like FAISS, but with built-in persistence to disk) or as a **client-server**. It handles metadata, persistence, and CRUD

cleanly with very little code. An excellent default for small-to-medium projects and the natural next step up from raw FAISS.

- **Pinecone** — a **fully managed, cloud-native** vector database. You never run a server; you call an API and Pinecone handles scaling, indexing, and availability. It scales to billions of vectors with low latency. It is a paid, proprietary service — you trade money and external hosting for zero operational burden.
- **Weaviate** — an open-source, feature-rich vector database (also offered as a managed cloud service). It has strong built-in support for hybrid search, metadata filtering, and modular integrations. A good choice when you want a powerful self-hostable database with batteries included.
- **Milvus** — an open-source vector database engineered for **massive scale**. Its distributed architecture is built to handle billions of vectors across many machines. It is the heavyweight option for very large deployments, with correspondingly more operational complexity.
- **Qdrant** — an open-source vector database (with a managed cloud option) written in Rust, known for **high performance and excellent metadata filtering**. A strong, modern choice that balances speed, rich filtering, and reasonable operational simplicity.

Database	Type	Hosting	Persistence	Scales to	Best for
FAISS	Library (in-process)	Self, in-app	Manual save/load	Millions	Prototyping, embedded search, learning
ChromaDB	Lightweight DB	Self (embedded or server)	Built-in	Millions	Small-medium projects; easy default
Pinecone	Managed cloud DB	Provider-managed	Built-in	Billions	Production with minimal ops effort
Weaviate	Open-source DB	Self or managed	Built-in	Billions	Feature-rich self-hosted deployments
Milvus	Open-source DB	Self or managed	Built-in	Billions+	Very large, distributed deployments
Qdrant	Open-source DB	Self or managed	Built-in	Billions	High performance + rich filtering

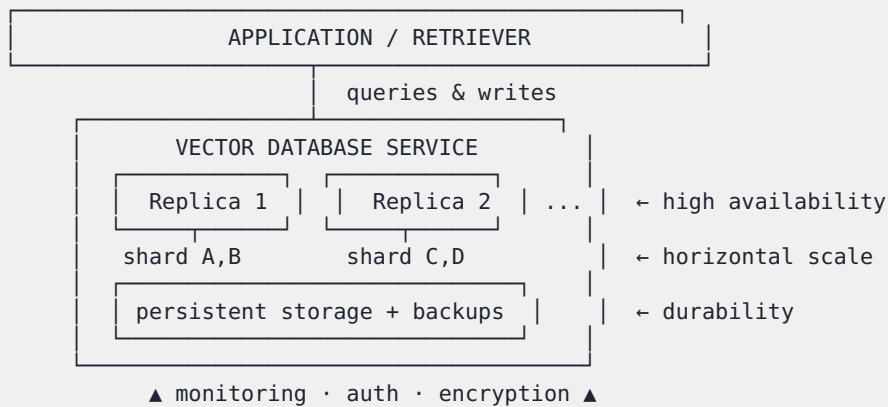
INTERVIEW INSIGHT

"Which vector database would you choose, and why?" is a question about *reasoning*, not about naming a favourite. A strong answer states that there is no universal best and walks the trade-offs: FAISS or Chroma for a prototype because they are free and zero-to-low setup; a managed service like Pinecone when the team wants production scale without operational burden; a self-hosted open-source database like Weaviate, Milvus, or Qdrant when data must stay in-house, when cost at scale matters, or when specific features (rich filtering, hybrid search, massive distributed scale) are needed. The dimensions to weigh aloud: scale, cost, operational effort, data-privacy requirements, filtering and hybrid-search needs, and latency.

4.10 Production Vector Database Architecture

A vector store in a real production system is more than "a place that holds vectors." A production-grade vector database deployment has several concerns layered on top of the core search:

- **Persistence and durability.** The index must survive restarts and machine failures, with backups and a recovery path. An in-memory-only store with no saved copy is a single crash away from losing hours of embedding work.
- **Scalability.** As the corpus grows, the database must scale — vertically (a bigger machine) or, for the largest systems, horizontally by **sharding** the vectors across multiple machines and merging their results.
- **High availability.** Production systems run **replicas** so that if one node fails, queries are still served. A managed service provides this for you; a self-hosted one you must configure.
- **Metadata indexing and filtering.** Beyond vector search, the database needs efficient filtering on metadata fields — which means those fields themselves need indexing.
- **Concurrency.** Many users query simultaneously while ingestion jobs write new vectors. The database must handle concurrent reads and writes safely.
- **Security and access control.** Authentication, encryption in transit and at rest, and (for multi-tenant systems) strict isolation so one tenant's queries can never reach another tenant's vectors.
- **Monitoring.** Query latency, recall, index size, and error rates must be observable so problems are caught before users feel them.

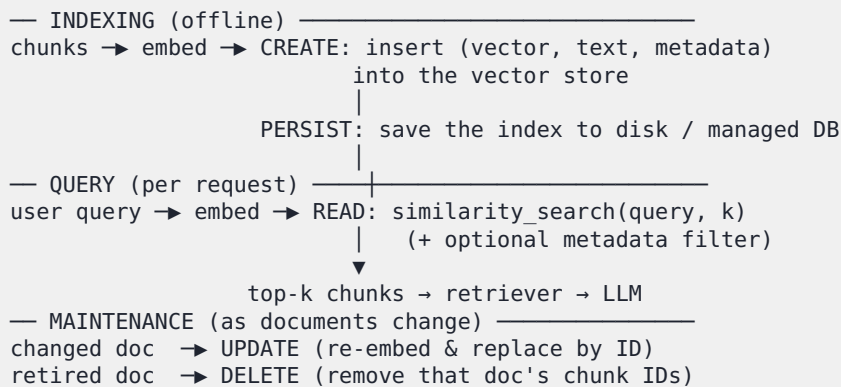


WHY THIS MATTERS

The gap between a prototype and a production RAG system is largely the gap described in this section. FAISS in a notebook is wonderful for learning — but it has no replicas, no built-in backups, no concurrency layer, no access control. The decision to use a managed service such as Pinecone, or to self-host Weaviate, Milvus, or Qdrant, is really a decision about *who builds and operates all of the above*. This is precisely why "we built a great prototype but productionising it is hard" is such a common story — and why understanding production architecture is what makes you valuable.

4.11 Workflow

The vector store is the meeting point of the indexing phase and the query phase:



The discipline that makes this workflow production-grade: **persist the index** so embedding work is never lost; **load the persisted index at query time** rather than rebuilding it; and **maintain the store incrementally** with targeted updates and deletes instead of rebuilding from scratch whenever a document changes.

4.12 Step-by-Step Implementation and Code

We will build two versions. First, FAISS — fast, in-memory, with explicit save and load. Then ChromaDB — persistent, with a full CRUD demonstration. Save the following as `module4_vectorstore.py`.

```

"""
module4_vectorstore.py
Building, persisting, searching, and maintaining a vector store.
Stack: Python + LangChain + Gemini + FAISS + Chroma.
Install: pip install langchain-community langchain-google-genai \
        langchain-chroma faiss-cpu python-dotenv
"""

import os
from dotenv import load_dotenv
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_chroma import Chroma
from langchain_core.documents import Document

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

# A tiny knowledge base for demonstration
docs = [
    Document(page_content="Full-time staff receive 24 days of paid annual leave.",
              metadata={"source": "handbook.pdf", "topic": "leave"}),
    Document(page_content="Reimbursement claims must be filed within 30 days.",
              metadata={"source": "handbook.pdf", "topic": "expenses"}),
    Document(page_content="The office is closed on all national holidays.",
              metadata={"source": "handbook.pdf", "topic": "holidays"}),
]

# ===== PART A: FAISS (in-memory + save/load) =====

# --- CREATE: build the store from documents ---
faiss_store = FAISS.from_documents(docs, embeddings)
print(f"[FAISS] Store created with", faiss_store.index.ntotal, "vectors.")

# --- PERSIST: save the index to disk so embedding work is not lost ---
faiss_store.save_local("faiss_index")
print(f"[FAISS] Index saved to ./faiss_index")

# --- LOAD: reuse the saved index without re-embedding ---
faiss_store = FAISS.load_local(
    "faiss_index", embeddings, allow_dangerous_deserialization=True
)

# --- READ: similarity search with scores ---
results = faiss_store.similarity_search_with_score("How many vacation days?", k=2)
print(f"\n[FAISS] Search: 'How many vacation days?'")
for doc, score in results:
    print(f"    score={score:.4f} | {doc.page_content}")

# ===== PART B: CHROMA (persistent + full CRUD) =====

# --- CREATE: a persistent Chroma store with explicit IDs ---
chroma_store = Chroma(
    collection_name="handbook",
    embedding_function=embeddings,
    persist_directory="chroma_db",      # persisted to disk automatically
)
chroma_store.add_documents(docs, ids=["leave-1", "exp-1", "hol-1"])
print(f"\n[CHROMA] Created collection with 3 documents.")

# --- READ ---
hits = chroma_store.similarity_search("rules about expense claims", k=1)
print(f"[CHROMA] Read:", hits[0].page_content)

# --- READ with metadata filter ---
filtered = chroma_store.similarity_search(
    "company rules", k=1, filter={"topic": "holidays"}
)

```

```

)
print("[CHROMA] Filtered read (topic=holidays):", filtered[0].page_content)

# --- UPDATE: the leave policy changed from 24 to 26 days ---
revised = Document(
    page_content="Full-time staff receive 26 days of paid annual leave.",
    metadata={"source": "handbook.pdf", "topic": "leave"},
)
chroma_store.update_document(document_id="leave-1", document=revised)
print("[CHROMA] Updated 'leave-1' to the new policy.")

# --- DELETE: remove the holidays record ---
chroma_store.delete(ids=["hol-1"])
print("[CHROMA] Deleted 'hol-1'.")

# --- Verify the update and delete took effect ---
check = chroma_store.similarity_search("annual leave days", k=1)
print("[CHROMA] After update:", check[0].page_content)

```

4.13 Line-by-Line Code Explanation

Setup. As before, `load_dotenv()` loads the Gemini key and the embedding model is created once. The three `Document` objects are a miniature knowledge base; in a real system these come from Module 2's `ingest.py`.

Part A — FAISS.

`FAISS.from_documents(docs, embeddings)` is the **Create** operation. It embeds every document and builds an in-memory FAISS index. `faiss_store.index.ntotal` confirms how many vectors the index now holds.

`faiss_store.save_local("faiss_index")` is **persistence** — it writes the index and the associated text/metadata to a folder on disk. This is the single line that turns minutes of embedding work into a reusable artifact.

`FAISS.load_local("faiss_index", embeddings, ...)` reloads that saved index in a fresh run, with **no re-embedding**. The `allow_dangerous_deserialization=True` flag is required because loading a FAISS index unpickles a file; only ever load index files you created or fully trust — never an index file from an untrusted source.

`faiss_store.similarity_search_with_score("How many vacation days?", k=2)` is the **Read** operation. Note the query says "vacation days" while the stored chunk says "paid annual leave" — different words, same meaning — and semantic search still ranks that chunk first. The returned score lets you judge *how* confident the match is, which feeds the thresholds of Module 5.

Part B — Chroma.

`Chroma(collection_name=..., embedding_function=..., persist_directory="chroma_db")` creates a **persistent** Chroma store. Unlike FAISS, Chroma writes to `persist_directory` automatically — there is no separate save step. A *collection* is Chroma's named container of vectors.

`chroma_store.add_documents(docs, ids=[...])` is **Create**, and crucially it supplies explicit, meaningful IDs. Those IDs are the stable handles that make update and delete precise — exactly the best practice from Section 4.8.

`chroma_store.similarity_search(...)` is **Read**. The second read adds `filter={"topic": "holidays"}`, demonstrating metadata filtering: the similarity search runs only over chunks whose `topic` metadata is `holidays`.

`chroma_store.update_document(document_id="leave-1", document=revised)` is **Update**. The leave policy changed from 24 to 26 days; because the *text* changed, Chroma re-embeds the revised document and replaces the record stored under ID `leave-1`. The old vector is gone; the new meaning is now searchable.

`chroma_store.delete(ids=["hol-1"])` is **Delete** — it removes the holidays record entirely.

The final search verifies the maintenance worked: querying for "annual leave days" now returns the *26-day* text, proving the update replaced the old chunk rather than duplicating it.

BEST PRACTICE

In a real application, the FAISS "create + save" code belongs in your *ingestion* script (run when documents change), and the "load + search" code belongs in your *query service* (run at startup, then reused for every request). Never rebuild the index inside the request path. This is the same indexing-versus-query separation stressed since Module 1 — the vector store is simply where that separation becomes a concrete pair of scripts.

4.14 Common Mistakes

- **Never persisting the index.** An in-memory store lost on restart means re-embedding the whole corpus every run — wasted time and API quota. Always save and reload.
- **Rebuilding the index inside the query path.** Building the store on every request is catastrophic for latency. Build once; load at startup.
- **Ignoring index configuration, then blaming the LLM.** An index tuned too aggressively for speed silently lowers recall. Weak retrieval can be an index problem, not a model problem.
- **Using random insert-time IDs.** Without stable, deterministic IDs, updates and deletes cannot target the right record, and re-ingestion creates duplicates.
- **Assuming you can edit a vector in place.** Changing a chunk's text changes its meaning; an update must re-embed. Update = delete + re-insert.
- **Forgetting to store the text and metadata.** A vector alone is not human-readable and carries no source. Store the triple `(vector, text, metadata)`.

- **Choosing a heavyweight database for a tiny project (or vice versa).** Match the database to the scale: FAISS or Chroma for small projects; a managed or distributed database only when scale genuinely demands it.
- **Loading an untrusted FAISS index file.** Deserialization can execute code; only load index files you created or fully trust.

4.15 Best Practices

- **Always persist the index** and reload it; never re-embed unnecessarily.
- **Separate build-and-save (ingestion) from load-and-search (serving).**
- **Use stable, deterministic IDs** so updates, deletes, and re-ingestion are precise and duplicate-free.
- **Maintain the store incrementally** — targeted updates and deletes — instead of full rebuilds.
- **Store text and metadata alongside every vector** for readable results and citations.
- **Match the database to the workload** along scale, cost, ops effort, privacy, and feature needs.
- **Understand your index's parameters** (`nprobe` , `ef`) as the speed-versus-recall dial, and tune them deliberately.
- **For production, plan for persistence, availability, scaling, security, and monitoring** — not just core search.

4.16 Real-World Applications

- **The RAG knowledge base** — the persistent, searchable store behind every retriever in this book.
- **Semantic search engines** — site-wide or app-wide "search by meaning" over documents, products, or media.
- **Recommendation systems** — store item embeddings; recommend the nearest neighbours to what a user liked.
- **Image, audio, and video search** — the same store, holding embeddings of non-text media, enabling "find similar images."
- **Deduplication and record matching** — near-identical records surface as near-identical vectors.
- **Anomaly and fraud detection** — events whose vectors sit far from every dense cluster are flagged as outliers.
- **Long-term memory for AI agents** — agents store past interactions as vectors and retrieve relevant memories later (a direct bridge to Module 8).

4.17 Mini Project: A Persistent, Maintainable Knowledge Base

Combine this module with the earlier ones into a small but real system:

1. Use Module 2's `ingest.py` to turn a folder of PDFs into chunks, and Module 3's embedding helper to prepare for embedding.
2. Build a **persistent Chroma store** from those chunks, assigning each chunk a **deterministic ID** of the form `filename::chunk_index`.
3. Write a `search(query)` function that loads the persisted store and prints the top-4 chunks with their similarity scores and `source` metadata.
4. **Test persistence.** Run the build script once, then run a separate search script. Confirm the search works *without* re-embedding anything.
5. **Test update.** Edit one PDF (change a fact), re-run ingestion for that file only, and use deterministic IDs so its chunks are overwritten rather than duplicated. Confirm a query now returns the corrected fact.
6. **Test delete.** Remove one document entirely: find every chunk ID whose `source` matches that file and delete them as a set. Confirm queries about that document now return nothing relevant.
7. **Inspect the index.** Print how many vectors the store holds before and after each operation, so you can *see* create, update, and delete changing the knowledge base.

This mini project is a genuine, maintainable RAG knowledge base — the durable foundation that Modules 5 onward will search and reason over.

4.18 Chapter Summary

- A **vector store** is a database specialised for storing embeddings and performing fast **similarity search**. Each record is a triple: **vector, original text, and metadata**.
- Unlike a traditional database (exact-match lookups), a vector store searches by **nearness** in the embedding space.
- **Brute-force (flat) search** compares the query against every vector — exact, but its cost grows linearly and it fails to scale.
- **Approximate Nearest Neighbour (ANN)** search trades a small, controllable loss of accuracy (**recall**) for a massive gain in speed by examining only part of the database. **Indexing** is the data structure that enables it. Every index juggles a three-way trade-off: **speed, recall, memory**.
- **IVF** clusters the vector space into zones and, at query time, searches only the zones nearest the query (tuned by `nprobe`). Memory-efficient; strong for very large, static datasets.

- **HNSW** builds a multi-layer navigable graph and walks from sparse long-range layers down to a dense local layer (tuned by `M` and `ef`). Excellent recall and speed; uses more memory; the common production default.
- Vector stores support **CRUD**: Create (insert), Read (similarity search), Update (re-embed and replace — you cannot edit a vector in place), and Delete (remove by ID). Stable, deterministic **IDs** make update and delete precise.
- The six databases to know: **FAISS** (in-process library, great for prototyping), **ChromaDB** (lightweight, persistent, easy default), **Pinecone** (fully managed cloud), **Weaviate** (feature-rich open-source), **Milvus** (massive-scale open-source), **Qdrant** (high-performance open-source). Choose by scale, cost, ops effort, privacy, and feature needs.
- A **production** vector database deployment adds persistence and durability, scalability (sharding), high availability (replicas), metadata indexing, concurrency, security, and monitoring on top of core search.
- You built and persisted real vector stores in FAISS and Chroma, and performed a full create-read-update-delete cycle.

4.19 Practice Exercises

1. **Conceptual.** Explain to a non-technical person, in under 120 words, why a vector store needs an "index" and what it gains and gives up by having one.
2. **Trade-off.** Define recall in one sentence. Then explain why a recall of 0.97 is usually perfectly acceptable for a RAG retriever, referring to how many chunks RAG retrieves and what the LLM does with them.
3. **IVF vs HNSW.** In your own words, contrast how IVF and HNSW *decide which vectors to examine*. Name the main parameter each exposes and say what turning it does.
4. **Database choice.** Recommend a vector database, with reasons, for each: (a) a weekend prototype on a laptop; (b) a startup that wants production scale with a tiny ops team; (c) an enterprise that may not send any data outside its own network and expects billions of vectors.
5. **Code — persistence.** Run Part A of `module4_vectorstore.py`, then write a *separate* script that only loads the saved FAISS index and searches it. Confirm no embedding happens on the second run.
6. **Code — CRUD.** Extend Part B: add a fourth document, search for it, update it, search again, then delete it and confirm it is gone. Print the vector count after each step.
7. **Investigation.** Using `similarity_search_with_score`, run five queries and record the scores. Identify a "good match" score range and a "weak match" range for your data. Explain how you would use this to set a score threshold (a preview of Module 5).

4.20 Interview Questions

BEGINNER

Q1. What is a vector store, and how does it differ from a traditional database?

A vector store is a database specialised for storing embeddings and finding the vectors most *similar* to a query vector. A traditional database searches by exact match on structured fields; a vector store searches by *nearness* in the embedding space. Each vector-store record bundles the vector, the original text, and metadata.

Q2. What three things does a vector store keep for each chunk?

The vector (the embedding, used for searching), the original text (returned and fed to the LLM), and the metadata (source, page, and so on, used for citations and filtering).

Q3. What are the four CRUD operations for a vector store?

Create (insert new vectors), Read (similarity search), Update (re-embed a changed chunk and replace its record), and Delete (remove records by ID).

Q4. Name three vector databases and one fact about each.

For example: FAISS — an in-process library, great for prototyping; ChromaDB — a lightweight persistent database, an easy default; Pinecone — a fully managed cloud service that handles scaling for you.

INTERMEDIATE

Q5. What is brute-force search and why does it not scale?

Brute-force (flat) search compares the query against every stored vector. It is exact, but its cost grows linearly with the number of vectors, so at millions of vectors each query becomes too slow for interactive use.

Q6. What is Approximate Nearest Neighbour search, and what does it trade?

ANN finds vectors very likely to be among the true nearest neighbours while examining only a small fraction of the database. It trades a small, controllable loss of accuracy — measured by recall — for a very large gain in speed. Indexing is the data structure that makes ANN possible.

Q7. Why can't you update a vector in place?

Because a chunk's vector encodes the meaning of its text. If the text changes, the meaning changes, so the vector must be recomputed by the embedding model. An update is therefore a re-embed and replace — conceptually a delete followed by an insert under the same ID.

Q8. Why are stable, deterministic IDs important in a vector store?

They give each record a reliable handle, so updates and deletes can target exactly the right vector, and re-ingesting a changed document overwrites its old chunks instead of creating duplicates. Random insert-time IDs make the knowledge base hard to maintain correctly.

ADVANCED**Q9. Explain the difference between IVF and HNSW indexing.**

IVF is cluster-based: it partitions the vector space into clusters (via k-means), and at query time compares the query only against the cluster centroids, then searches the few nearest clusters (controlled by `nprobe`). It is memory-efficient and suits very large, static datasets. HNSW is graph-based: it builds a multi-layer proximity graph and navigates from sparse, long-range upper layers down to a dense local layer (tuned by `M` and `ef`). It gives excellent recall and speed but uses more memory; it is the common production default. Both are ANN methods trading a little recall for a lot of speed.

Q10. What is the three-way trade-off in vector indexing, and how do you navigate it?

Speed, recall (accuracy), and memory — you cannot maximise all three. You navigate it by choosing an index type suited to your data size and update pattern, then tuning its parameters (`nprobe` for IVF, `ef` for HNSW) toward more speed or more recall, validated by measuring recall against an exact search on a sample.

Q11. What does a production vector database deployment need beyond core similarity search?

Persistence and durability with backups; scalability, including sharding vectors across machines for the largest corpora; high availability via replicas; efficient metadata indexing and filtering; safe concurrency for simultaneous reads and writes; security — authentication, encryption, and tenant isolation; and monitoring of latency, recall, and errors. A managed service provides much of this; a self-hosted database means you build and operate it.

SCENARIO-BASED**Q12. A RAG prototype built on FAISS works well in a notebook, but the team cannot turn it into a reliable product. What is likely missing, and what would you recommend?**

FAISS is an in-process library with no built-in persistence guarantees, no replicas, no concurrency layer, no access control, and no monitoring — everything a production service needs. The prototype is fine; the deployment story is missing. Recommend moving to a production-grade vector database: a managed service like Pinecone if the team wants to avoid operations, or a self-hosted open-source database (Weaviate, Milvus, Qdrant) if data must stay in-house or cost at scale matters. The application code changes little thanks to LangChain's uniform interface; what is gained is durability, availability, concurrency, security, and observability.

Q13. Users report that your RAG system "used to know" a fact that has since changed, and now gives the outdated answer. The document was corrected weeks ago. What went wrong, and how do you fix and prevent it?

The corrected document was re-ingested, but the *old* chunks were never removed — so the stale vectors still sit in the store and still get retrieved. The likely root cause is non-deterministic IDs: re-ingestion inserted new chunks alongside the old ones instead of overwriting them. Fix it now by deleting the stale chunks (find them by the document's `source` metadata). Prevent recurrence by adopting deterministic IDs so re-ingesting a document overwrites its chunks, and by making the ingestion pipeline delete a document's existing chunks before re-adding it — the incremental-update discipline from Module 2.

END OF MODULE 4 — AND THE INDEXING PHASE

With this module the entire **indexing phase** is complete: you can load documents, split them into chunks, embed those chunks, and store them in a fast, persistent, maintainable vector database. You now hold a real, searchable knowledge base. **Module 5 — Basic Retrieval Techniques** opens the **query phase**: similarity search in practice, score thresholds, Maximal Marginal Relevance for balancing relevance against diversity, hybrid search combining dense and sparse retrieval, and ensemble retrievers. This is where your knowledge base finally starts answering questions well.